

Ans. to Question No-01(a)

$$\begin{aligned}
 \text{i) } Y(5) &= Y(4) + B(5) \\
 &= Y(3) + B(4) + 6 \\
 &= Y(2) + B(3) + 5 + 6 \\
 &= Y(1) + B(2) + 4 + 11 \\
 &= Y(0) + 7 + 19 \\
 &= 0 + 16 \\
 &= 16
 \end{aligned}$$

$$\begin{aligned}
 \text{ii) } Y(5) &= Y(4) \\
 &= Y(3) + 11 \\
 &= Y(2) + B(3) + 11 \\
 &= Y(1) + B(2) + 4 + 11 \\
 &= Y(0) + 7 + 11 \\
 &= 0 + 3 \\
 &= 3
 \end{aligned}$$

Ans. to Question No-01(b)

$$\begin{aligned}
 1. A(2, 2) &= A(1, A(2, 1)) \\
 2. A(2, 1) &= A(1, A(2, 0)) \\
 3. A(2, 0) &= A(1, 1) \\
 4. A(1, 1) &= A(0, A(1, 0)) \\
 5. A(1, 0) &= A(0, 1) \\
 6. A(0, 1) &= 1+1 = 2 \\
 7. A(1, 0) &= 2 \\
 8. A(1, 1) &= A(0, 2)
 \end{aligned}$$

$$\begin{aligned}
 9. A(0, 2) &= 2+1 = 3 \\
 10. A(1, 1) &= 3 \\
 11. A(2, 0) &= 3 \\
 12. A(2, 1) &= A(1, 3) \\
 13. A(1, 3) &= A(0, A(1, 2)) \\
 14. A(1, 2) &= A(0, A(1, 1)) \\
 15. A(1, 1) &= A(0, A(1, 0)) \\
 16. A(1, 0) &= A(0, 1) \\
 17. A(0, 1) &= 1+1 = 2 \\
 18. A(1, 0) &= 2 \\
 19. A(1, 1) &= A(0, 2) \\
 20. A(0, 2) &= 2+1 = 3 \\
 21. A(1, 1) &= 3 \\
 22. A(1, 2) &= A(0, 3) \\
 &A(0, 3) = 3+1 = 4
 \end{aligned}$$

1. $A(2,2) = A(1, A(2,1))$
2. $A(2,1) = A(1, A(2,0))$
3. $A(2,0) = A(1,1)$
4. $A(1,1) = A(0, A(1,0))$
5. $A(1,0) = A(0,1)$
6. $A(0,1) = 1+1 = 2$
7. $A(1,0) = 2$
8. $A(1,1) = A(0,2)$

9. $A(0,2) = 2+1 = 3$
10. $A(1,1) = 3$
11. $A(2,0) = 3$
12. $A(2,1) = A(1,3)$
13. $A(1,3) = A(0, A(1,2))$
14. $A(1,2) = A(0, A(1,1))$
15. $A(1,1) = A(0, A(1,0))$
16. $A(1,0) = A(0,1)$
17. $A(0,1) = 1+1 = 2$
18. $A(1,0) = 2$
19. $A(1,1) = A(0,2)$
20. $A(0,2) = 2+1 = 3$
21. $A(1,1) = 3$
22. $A(1,2) = A(0,3)$
23. $A(0,3) = 3+1 = 4$
24. $A(1,2) = 4$
25. $A(1,3) = A(0,4)$
26. $A(0,4) = 4+1 = 5$
27. $A(1,3) = 5$
28. $A(2,1) = 5$
29. $A(2,2) = A(1,5)$
30. $A(1,5) = A(0, A(1,4))$
31. $A(1,4) = A(0, A(1,3))$
32. $A(1,3) = A(0, A(1,2))$
33. $A(1,2) = A(0, A(1,1))$
34. $A(1,1) = A(0, A(1,0))$
35. $A(1,0) = A(0,1)$
36. $A(0,1) = 1+1 = 2$

37. $A(1,0) = 2$
38. $A(1,1) = A(0,2)$
39. $A(0,2) = 2+1 = 3$
40. $A(1,1) = 3$
41. $A(1,2) = A(0,3)$
42. $A(0,3) = 3+1 = 4$
43. $A(1,2) = 4$
44. $A(1,3) = A(0,4)$
45. $A(0,4) = 4+1 = 5$
46. $A(1,3) = A(0,5)$

26. $A(1, 3) = 5$
 27. $A(2, 1) = 5$
 28. $A(2, 2) = A(1, 5)$
 29. $A(1, 5) = A(0, A(1, 4))$
 30. $A(1, 4) = A(0, A(1, 3))$
 31. $A(1, 3) = A(0, A(1, 2))$
 32. $A(1, 2) = A(0, A(1, 1))$
 33. $A(1, 1) = A(0, A(1, 0))$
 34. $A(1, 0) = A(0, 1)$
 35. $A(0, 1) = 1 + 1 = 2$
 36.

37. $A(1, 0) = 2$
 38. $A(1, 1) = A(0, 2)$
 39. $A(0, 2) = 2 + 1 = 3$
 40. $A(1, 1) = 3$
 41. $A(1, 2) = A(0, 3)$
 42. $A(0, 3) = 3 + 1 = 4$
 43. $A(1, 2) = 4$
 44. $A(1, 3) = A(0, 4)$
 45. $A(0, 4) = 4 + 1 = 5$
 46. $A(1, 3) = A(0, 5)$
 47. $A(0, 5) = 5 + 1 = 6$
 48. $A(1, 4) = 6$
 49. $A(1, 5) = A(0, 6)$
 50. $A(0, 6) = 6 + 1 = 7$
 51. $A(1, 5) = 7$
 52. $A(2, 2) = 7$

Ans: 7.

Ans. to Question No-01 (b) on

1. $5! = 5 \cdot 4!$
 2. $4! = 4 \cdot 3!$
 3. $3! = 3 \cdot 2!$
 4. $2! = 2 \cdot 1!$
 5. $1! = 1 \cdot 0!$
 6. $0! = 1$
 7. $1! = 1 \cdot 1 = 1$
 8. $2! = 2 \cdot 1 = 2$
 9. $3! = 3 \cdot 2 = 6$
 10. $4! = 4 \cdot 6 = 24$

10. $5! = 5 \cdot 24 = 120$

Ans. to Question No-01 (c)

- i. $REAR - FRONT + 1 = 10 - 5 + 1 = 6$
 ii. $REAR + N - FRONT + 1 = 3 + 12 - 12 + 1 = 4$
 iii. $REAR - FRONT + 1 = 6 - 6 + 1 = 1$

34.
35.
36.

$$A(1,0) = A(0,1)$$

$$A(0,1) = 1+1 = 2$$

37. $A(1,0) = 2$
 38. $A(1,1) = A(0,2)$
 39. $A(0,2) = 2+1 = 3$
 40. $A(1,1) = 3$
 41. $A(1,2) = A(0,3)$
 42. $A(0,3) = 3+1 = 4$
 43. $A(1,2) = 4$
 44. $A(1,3) = A(0,4)$
 45. $A(0,4) = 4+1 = 5$
 46. $A(1,4) = A(0,5)$
 47. $A(0,5) = 5+1 = 6$
 48. $A(1,4) = 6$
 49. $A(1,5) = A(0,6)$
 50. $A(0,6) = 6+1 = 7$
 51. $A(1,5) = 7$
 52. $A(2,2) = 7$

Ans: 7.

Ans. to Question No-01 (b) on

1. $5! = 5 \cdot 4!$
 2. $4! = 4 \cdot 3!$
 3. $3! = 3 \cdot 2!$
 4. $2! = 2 \cdot 1!$
 5. $1! = 1 \cdot 0!$
 6. $0! = 1$
 7. $1! = 1 \cdot 1 = 1$
 8. $2! = 2 \cdot 1 = 2$
 9. $3! = 3 \cdot 2 = 6$
 10. $4! = 4 \cdot 6 = 24$

10. $5! = 5 \cdot 24 = 120$

Ans. to Question No-01 (c)

- i. $REAR - FRONT + 1 = 10 - 5 + 1 = 6$
 ii. $REAR + N - FRONT + 1 = 3 + 12 - 12 + 1 = 4$
 iii. $REAR - FRONT + 1 = 6 - 6 + 1 = 1$

— • —

8.

$$3! = 3 \cdot 2 = 6$$

9.

$$4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$$

$$10. 5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Ans. to Question No-01 (c)

$$i. REAR - FRONT + 1 = 10 - 5 + 1 = 6$$

$$ii. REAR + N - FRONT + 1 = 3 + 12 - 12 + 1 = 4$$

$$iii. REAR - FRONT + 1 = 6 - 6 + 1 = 1$$

— • —

CSE-2321

Ans. to Question No-02(a)

Linked list:

A linked list is a linear data structure in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers.

Advantages and Disadvantages over Linear Array:

In linear Array, sequential memory is needed to store data whereas, in linked list, this is not mandatory. So, this is a great advantage of linked list. But in linked list more memory is needed because of storing the pointers to next node. On the other hand, there is no direct access to middle element of array.

Ans. to Question No-02(a)Linked List:

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers.

Advantages and Disadvantages over Linear Array:

In linear Array, sequential memory is needed to store data whereas, in linked list, this is not mandatory. So, this is a great advantage of linked list. But in linked list more memory is needed because of storing the pointers to next node. On the other hand, there is no direct access to middle element of any range. For that reason, binary search algorithm can't be implemented whereas it can be implemented on linear array.

Ans. to Question No-02(b)

Step 1: Set PTR := START.

Step 2: Repeat step 3 while PTR $\neq$ NULL:

Step 3: If ITEM < INFO[PTR], then:

Set PTR := LINK[PTR].

Else if ITEM = INFO[PTR], then:

Set LOC := PTR, and Exit.

Else

Set LOC := NULL, and Exit.

[End of If structure]

[End of Step 2 loop]

Step 4: Set LOC := NULL.

Step 5: Exit.

Ans. to Question No-02(c)

After adding 12

	Info	Link
1	12	0
2		5
3	41	11
4	129	12
5		0
6	9	3
7	123	4
8	78	7

Avail $\rightarrow$ 2

start $\rightarrow$ 6

Step 1: Set PTR := START.

Step 2: Repeat step 3 while PTR $\neq$ NULL:

Step 3: If ITEM $<$ INFO[PTR], then:

Set PTR := LINK[PTR].

Else if ITEM = INFO[PTR], then:

Set LOC := PTR, and Exit.

Else

set LOC := NULL, and Exit.

[End of If structure]

[End of Step 2 loop]

Step 4: Set LOC := NULL.

Step 5: Exit.

Ans. to Question No-02 (c)

After adding 12

	Info	Link
1	12	0
2		5
3	41	11
4	129	12
5		0
6	9	3
7	123	4
8	78	7
9	194	10
10	231	1
11	62	8
12	145	9

Avail $\rightarrow$ 2

start $\rightarrow$ 6

after adding 12 and
then 194 is deleted
from the list then

	Info	Link
1	12	0
2		5
3	41	11
4	129	12
5		0
6	9	3
7	123	4
8	78	7
9		2
10	231	1

start $\rightarrow$ 6

Avail $\rightarrow$ 9

after adding 12 and
then 104 is deleted
from the list then

	Info	Link
1	12	0
2		5
3	41	11
4	129	12
5		0
start → 6	9	3
7	123	4
8	78	7
Avail → 9		2
10	231	3
11	62	8
12	145	10

Am. to Question No-02(d)

Two way linked list:

A two way list is a linear collection of data elements, called nodes, where each node N is divided into 3 parts:

1. An information field which contains the data of N.
2. A pointer field Forw which contains the location of the next node in the list.
3. A pointer field Back which contains the location of the preceding node in the list.

	Info	FORW	BACK
Avail → 1		2	
2		5	
3	41	11	6
4	129	12	7
5		0	
First → 6	9	3	0
7	123	4	8
8	78	7	11

- A two way list is a linear collection of data elements, called nodes, where each node N is divided into 3 parts:
1. An information field which contains the data of N .
 2. A pointer field $Forw$ which contains the location of the next node in the list.
 3. A pointer field $Back$ which contains the location of the preceding node in the list.

	Info	FORW	BACK
Avail → 1 [1]		2	
2		5	
3	41	11	6
4	129	12	7
5		0	
First → 6 [6]	9	3	0
7	123	4	8
8	78	7	11
9	104	10	12
Last → 10 [10]	231	0	9
11	62	9	3
12	145	9	4

Answer to the Question no 3(a)

3(a)	22	33	29	49	21	57	62	73	54	44
Passes										
A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]	
k=1, L=5	(22)	33	29	49	(21)	57	62	73	54	44
k=2, L=5	21	(33)	29	49	(22)	57	62	73	54	44
k=3, L=9	21	22	(29)	49	33	62	62	73	54	44
k=4, L=5	21	22	29	(49)	(33)	57	62	73	54	44
k=5, L=10	21	22	29	33	(49)	57	62	73	54	(44)
k=6, L=10	21	22	29	33	44	(57)	62	73	54	(49)
k=7, L=9	21	22	29	33	44	49	(62)	73	(54)	57
k=8, L=10	21	22	29	33	44	49	54	(73)	62	(57)
Sorted:	21	22	29	33	44	49	54	57	62	73

Answer to the Question no 3(b)

Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

- a) Heap Sort
- b) Selection Sort
- c) Insertion Sort

Answer to the Question no 3(b)

Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

a)

Heap Sort

b)

Selection Sort

c)

Insertion Sort

d)

Merge Sort

Correct answer is option 'B'. Can you explain this answer?

Verified Answer

Consider a situation where swap operation is very costly. Which of the...

Here The answer depends on the which algorithm has minimum number of swaps among these

And Among these ,selection sort has the minimum number of swaping complexity = $O(n)$ always

so Selection sort is preferable here

Most Upvoted Answer

Consider a situation where swap operation is very costly. Which of the...

Sorting Algorithms with Minimum Swap Operations

When the swap operation is very costly, we need to choose a sorting algorithm that minimizes the number of swap operations. Among the given options, the preferred sorting algorithm is Selection Sort. Let's see why.

Selection Sort

Selection Sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the array and putting it at the beginning. It has the following steps:

number of swap operations. Among the given options, the preferred sorting algorithm is Selection Sort. Let's see why.

Selection Sort

Selection Sort is a simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted part of the array and putting it at the beginning. It has the following steps:

- Find the minimum element in the unsorted part of the array.
- Swap it with the first element of the unsorted part.
- Move the boundary of the sorted part one element to the right.

Advantages of Selection Sort

- Selection Sort performs the minimum number of swap operations among all the comparison-based sorting algorithms.
- It has a time complexity of $O(n^2)$ which is better than some other sorting algorithms like Bubble Sort.

Disadvantages of Selection Sort

- Selection Sort has a time complexity of $O(n^2)$ which is not efficient for large arrays.
- It does not perform well if the array has many duplicate elements.

Comparison with Other Sorting Algorithms

Heap Sort: Heap Sort is a comparison-based sorting algorithm that uses a heap data structure to sort an array. It has a time complexity of $O(n \log n)$ which is better than Selection Sort. However, Heap Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

Insertion Sort: Insertion Sort is a simple sorting algorithm that works by repeatedly inserting an element from the unsorted part into its correct position in the sorted part. It has a time complexity of $O(n^2)$ which is the same as Selection Sort. However, Insertion Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

- Find the minimum element in the unsorted part of the array.
- Swap it with the first element of the unsorted part.
- Move the boundary of the sorted part one element to the right.

Advantages of Selection Sort

- Selection Sort performs the minimum number of swap operations among all the comparison-based sorting algorithms.
- It has a time complexity of $O(n^2)$ which is better than some other sorting algorithms like Bubble Sort.

Disadvantages of Selection Sort

- Selection Sort has a time complexity of $O(n^2)$ which is not efficient for large arrays.
- It does not perform well if the array has many duplicate elements.

Comparison with Other Sorting Algorithms

Heap Sort: Heap Sort is a comparison-based sorting algorithm that uses a heap data structure to sort an array. It has a time complexity of $O(n \log n)$ which is better than Selection Sort. However, Heap Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

Insertion Sort: Insertion Sort is a simple sorting algorithm that works by repeatedly inserting an element from the unsorted part into its correct position in the sorted part. It has a time complexity of $O(n^2)$ which is the same as Selection Sort. However, Insertion Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

Merge Sort: Merge Sort is a comparison-based sorting algorithm that uses a divide-and-conquer strategy to sort an array. It has a time complexity of $O(n \log n)$ which is better than Selection Sort. However,

Merge Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

Merge Sort: Merge Sort is a comparison-based sorting algorithm that uses a divide-and-conquer strategy to sort an array. It has a time complexity of $O(n \log n)$ which is better than Selection Sort. However,

Merge Sort performs more swap operations than Selection Sort, which makes it less suitable when swap operations are very costly.

Conclusion

When swap operation is very costly, the preferred sorting algorithm is Selection Sort because it performs the minimum number of swap operations among all the comparison-based sorting algorithms. However, if the array is very large or has many duplicate elements, other sorting algorithms like Heap Sort or Merge Sort may be more suitable despite performing more swap operations.

Answer to the Question no 3(c)

Insertion Sort

Algorithm 9.1: (Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements.

1. Set $A[0] := -\infty$. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$. [Moves element forward.]
 - (b) Set $PTR := PTR - 1$.

[End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element in proper place.]

[End of Step 2 loop.]
6. Return.

Answer to the Question no 3(d)

When swap operation is very costly, the preferred sorting algorithm is Selection Sort because it performs the minimum number of swap operations among all the comparison-based sorting algorithms. However, if the array is very large or has many duplicate elements, other sorting algorithms like Heap Sort or Merge Sort may be more suitable despite performing more swap operations.

Answer to the Question no 3(c)

Insertion Sort

Algorithm 9.1: (Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements.

1. Set $A[0] := -\infty$. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$. [Moves element forward.]
 - (b) Set $PTR := PTR - 1$.
5. [End of loop.]
 Set $A[PTR + 1] := TEMP$. [Inserts element in proper place.]
 [End of Step 2 loop.]
6. Return.

Answer to the Question no 3(d)

Q 3(d) 25, 42, 96, 101, 102, 162, 197, ~~222~~
 222 (JD = 0221222)

Table - size = 11

$$H(K) = \text{key} \% 11$$

$$H(25) = 25 \% 11 = 3$$

$$H(42) = 42 \% 11 = 9$$

$$H(96) = 96 \% 11 = 8$$

$$H(101) = 101 \% 11 = 2$$

$$H(102) = 102 \% 11 = 3$$

As linear probing

$$H(K) = 3 + 1 = 4$$

$$H(162) = 162 \% 11 = 8$$

$$H(K) = 8 + 1 = 9 + 1 = 10$$

$$H(197) = 197 \% 11 = 10$$

$$\therefore H(K) = 10 + 1 = 11$$

$$H(222) = \cancel{222} \% 11 = 2$$

$$H(K) = 2 + 1 = 3 + 1 = 4 + 1 = 5$$

	1
101	2
25	3
102	4
222	5
	6
	7
96	8
42	9
162	10
197	11

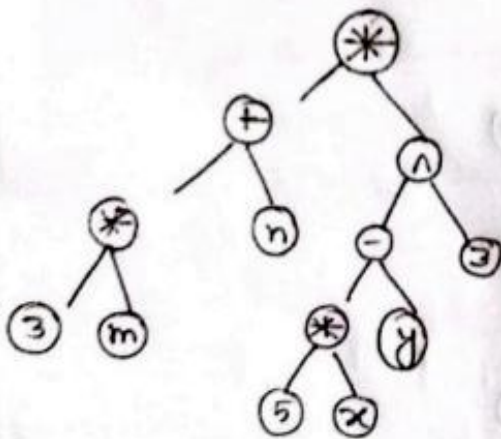
Answer to the Question no 4a(a,b)

Answer to the Question No-4(a)-(a)
Given that $(3m+n)(5x-y)^3$

Answer to the Question No-4(a)-(a)

Given that $(3m+n)(5x-y)^3$

Now drawing the tree T which corresponds to the given expression



Answer to the Question No 4(b)-(b)

The pre order of the tree T is

pre-order : $* + * 3 m n \wedge - * 5 x y 3$

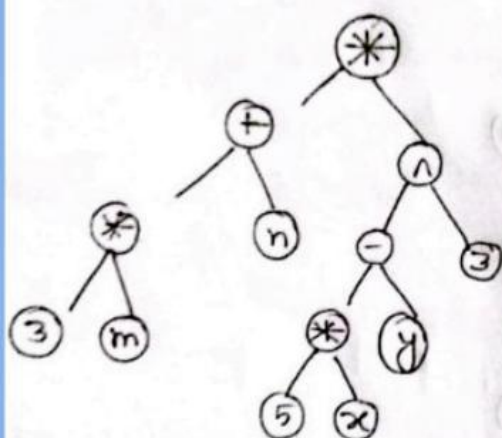
post order : $3 m * n + 5 x * y - 3 \wedge *$

Answer to the Question no 4a(a,b)

Answer to the Question No-4(a)-(a)

Given that $(3m+n)(5x-y)^3$

Now drawing the tree T which corresponds to the given expression



Answer to the Question No 4(b)-(b)

The pre order of the tree T is

pre-order : $* + * 3 m n \wedge - * 5 x y 3$

post order : $3 m * n + 5 x * y - 3 \wedge *$

Autumn-22

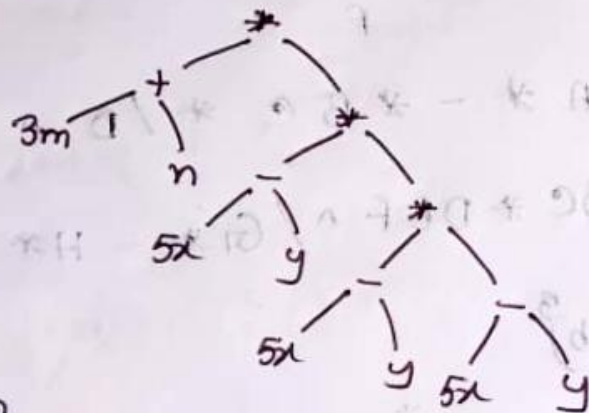
(40)

Consider the algebraic expression $E = (3m+n)(5x-y)^3$

(a) Draw the tree T which corresponds to the expression E

$$E = (3m+n) * (5x-y)^3$$

$$= (3m+n) * (5x-y) * (5x-y) * (5x-y)$$



(b) Find Preorder and postorder

Preorder :

(Root, L, R)

* + 3m n * - 5x y *
- 5x y - 5x y

Postorder: $3m \quad n + 5x \quad y - 5x \quad y - 5x \quad y - **$
 (L, R, Root) *

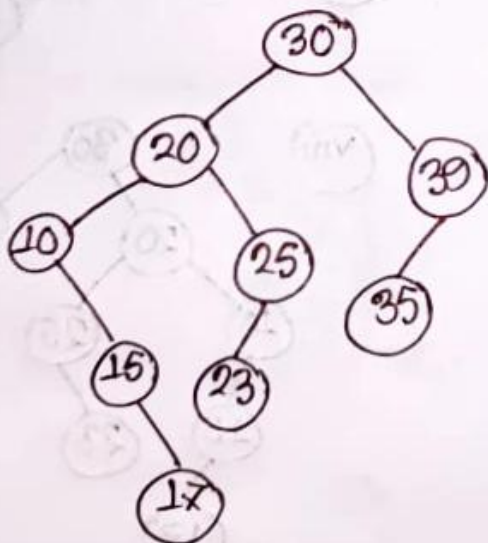
(46)

The Preorder Traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 17

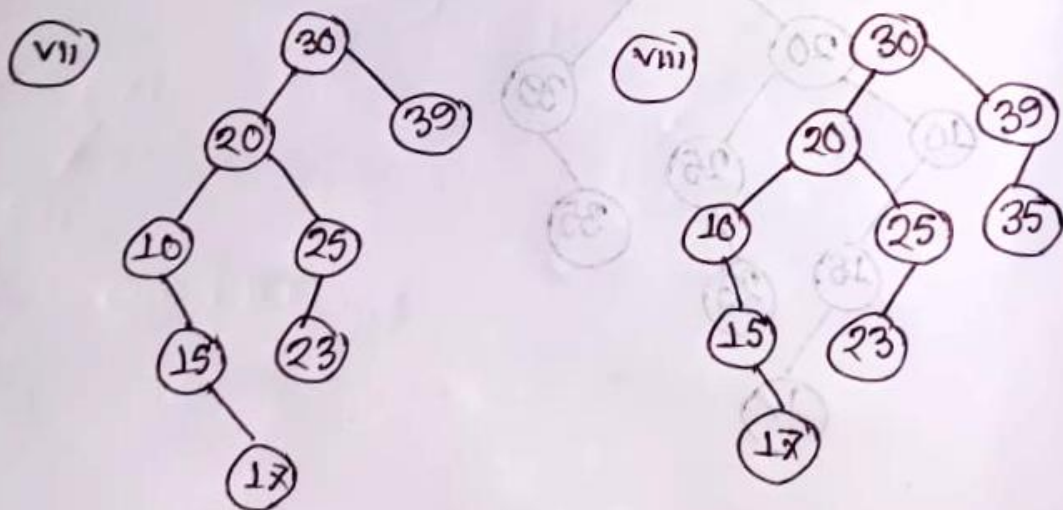
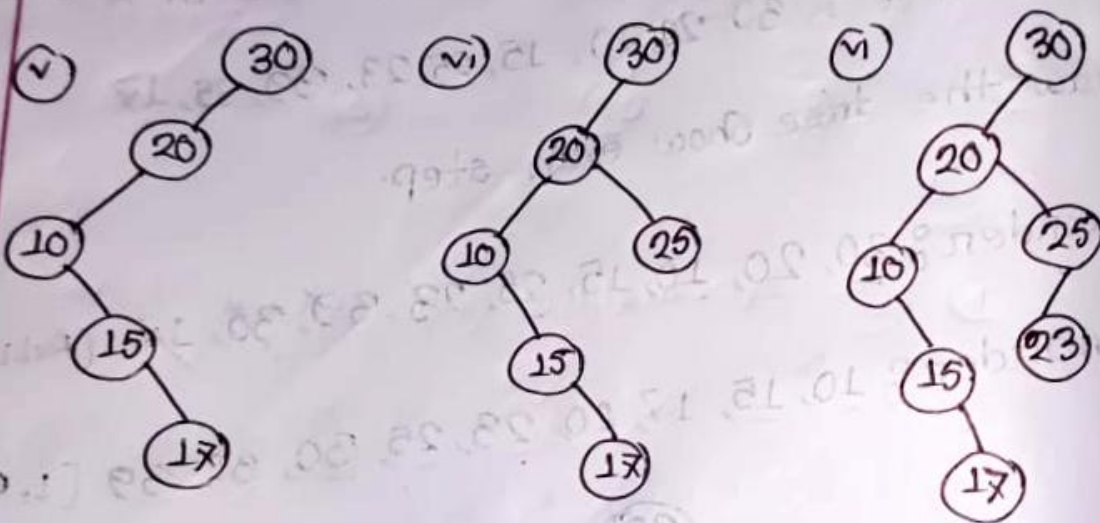
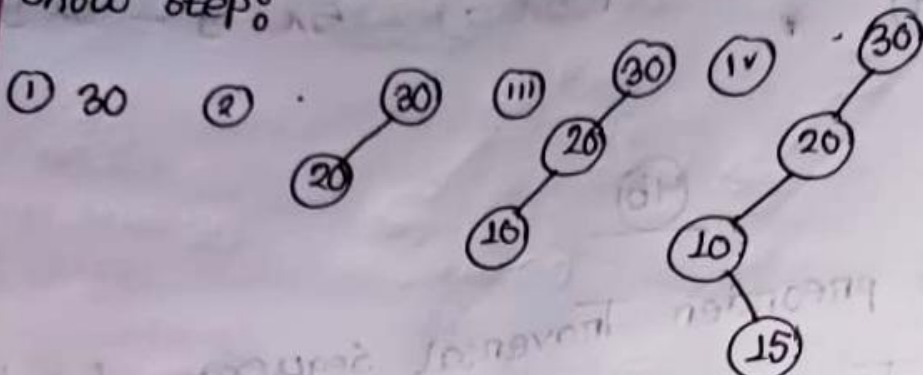
Draw the tree show each step.

Preorder: 30, 20, 10, 15, 25, 23, 39, 35, 17 [Root, L, R]

Inorder: 10, 15, 17, 20, 23, 25, 30, 35, 39 [L, Root, R]



Show step:



(40)

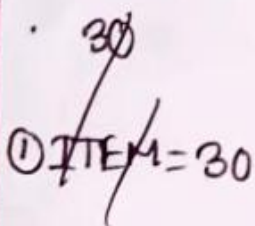
What do you mean by max heap? Build a max heap from the following list of numbers:

30, 39, 29, 27, 37, 52, 63, 44

⇒ Head or maxheap is a specialized tree-based data structure which is essentially an almost complete tree that satisfies the heap property.

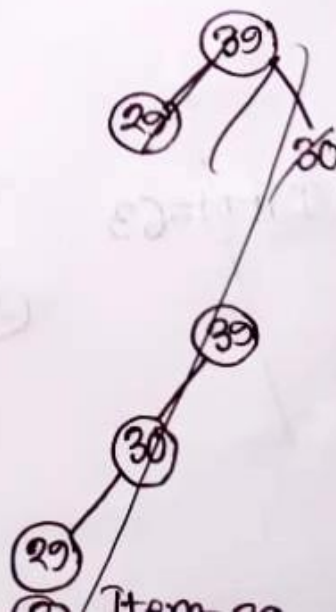
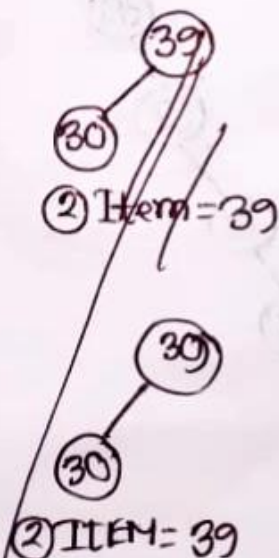
Maxheap → The value of N is greater than or equal to children.

Minheap → The value of N is less than children.



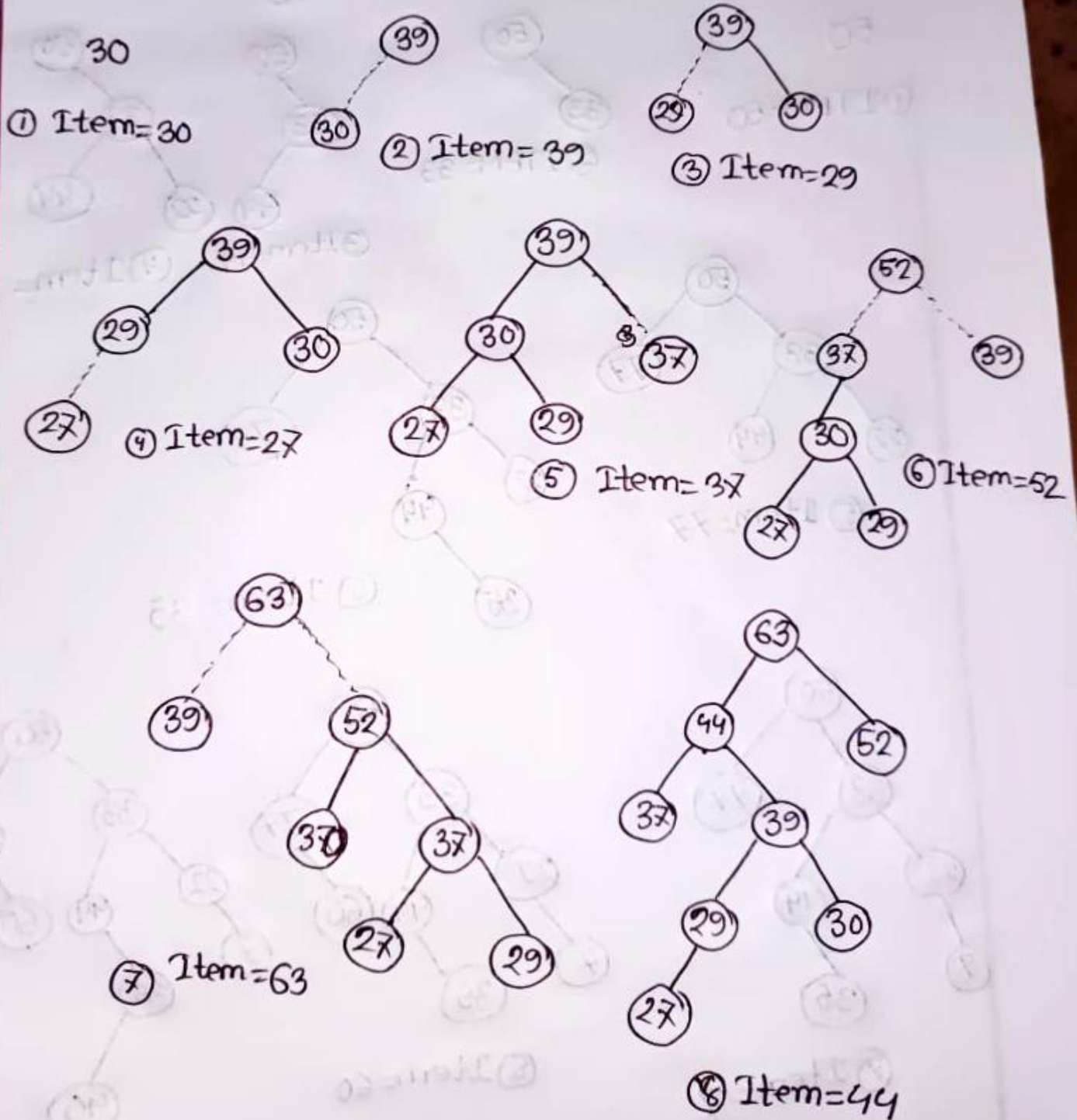
30

① Item = 30



(40)

30, 39, 29, 27, 37, 52, 63, 44

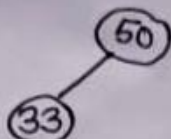


Qd on

50, 33, 44, 22, 77, 35, 7, 60, 40

50

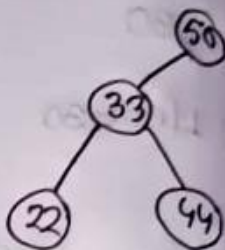
① Item=50



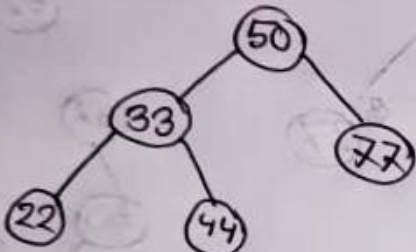
② Item=33



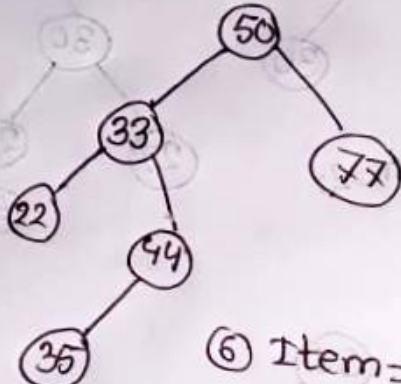
③ Item=44



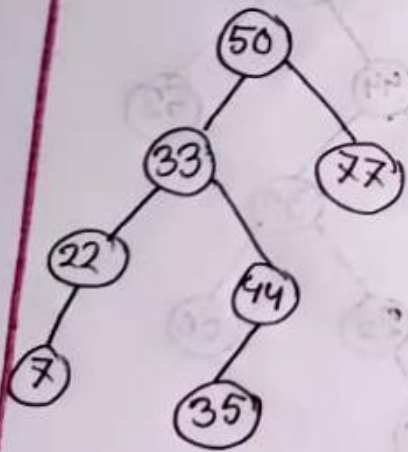
④ Item=22



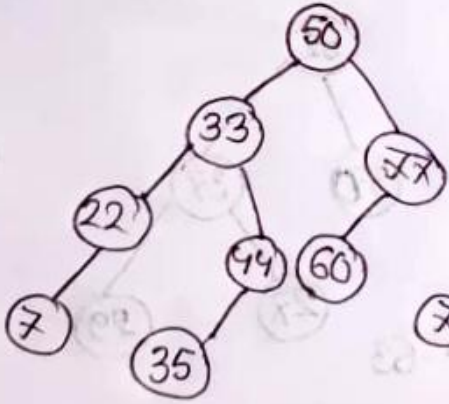
⑤ Item=77



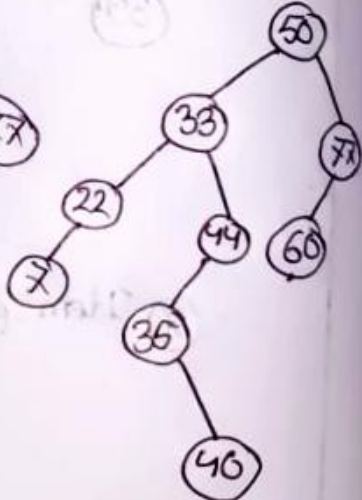
⑥ Item=35



⑦ Item=7



⑧ Item=60



⑨ Item=40

